```
// ── LIFE TRACKER · SERVICE WORKER v1.0 ──
const CACHE_NAME = 'lifetracker-v3';
const STATIC_ASSETS = [
  '/',
  '/index.html',
  '/dashboard.html',
  '/calendar.html',
  '/tasks.html',
  '/habits.html',
  '/finance.html',
  '/notes.html',
  '/assistant.html',
  '/profile.html',
  '/manifest.json'
];

// ── INSTALL: cachea los archivos estáticos ──
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => {
      return cache.addAll(STATIC_ASSETS).catch(() => {
        // Si algún archivo no existe aún, continúa igual
        return Promise.resolve();
      });
    })
  );
  self.skipWaiting();
});

// ── ACTIVATE: limpia caches viejos ──
self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(keys =>
      Promise.all(
        keys.filter(k => k !== CACHE_NAME).map(k => caches.delete(k))
      )
    )
  );
  self.clients.claim();
});

// ── FETCH: sirve desde cache si está offline ──
self.addEventListener('fetch', event => {
  // Solo intercepta peticiones GET al mismo origen
```

```
  if (event.request.method !== 'GET') return;
  if (!event.request.url.startsWith(self.location.origin)) return;

  event.respondWith(
    caches.match(event.request).then(cached => {
      if (cached) return cached;
      return fetch(event.request).then(response => {
        // Cachea páginas HTML y assets propios
        if (response && response.status === 200) {
          const clone = response.clone();
          caches.open(CACHE_NAME).then(cache => cache.put(event.request, clone));
        }
        return response;
      }).catch(() => {
        // Offline fallback para páginas HTML
        if (event.request.headers.get('accept')?.includes('text/html')) {
          return caches.match('/dashboard.html');
        }
      });
    })
  );
});

// ── PUSH NOTIFICATIONS ──
self.addEventListener('push', event => {
  let data = { title: '⏰ Life Tracker', body: 'Tienes un recordatorio pendiente.
  try {
    if (event.data) data = event.data.json();
  } catch(e) {
    if (event.data) data.body = event.data.text();
  }

  event.waitUntil(
    self.registration.showNotification(data.title, {
      body: data.body,
      icon: '/icon-192.png',
      badge: '/icon-72.png',
      tag: data.tag || 'lt-notification',
      renotify: true,
      vibrate: [200, 100, 200],
      actions: [
        { action: 'open', title: '✓ Ver' },
        { action: 'dismiss', title: 'Cerrar' }
      ],
      data: { url: data.url || '/dashboard.html' }
    })
  );
```

```
});

// ── NOTIFICATION CLICK ──
self.addEventListener('notificationclick', event => {
  event.notification.close();
  if (event.action === 'dismiss') return;

  const url = event.notification.data?.url || '/dashboard.html';
  event.waitUntil(
    clients.matchAll({ type: 'window', includeUncontrolled: true }).then(clientLi
      // Si ya hay una ventana abierta, la enfoca
      for (const client of clientList) {
        if (client.url.includes(self.location.origin) && 'focus' in client) {
          client.navigate(url);
          return client.focus();
        }
      }
      // Si no hay ventana abierta, abre una nueva
      if (clients.openWindow) return clients.openWindow(url);
    })
  );
});

// ── SCHEDULED NOTIFICATIONS (via postMessage) ──
// La app puede enviar mensajes para programar notificaciones
self.addEventListener('message', event => {
  if (event.data?.type === 'SCHEDULE_NOTIFICATION') {
    const { title, body, delay, tag, url } = event.data;
    setTimeout(() => {
      self.registration.showNotification(title || '⏰ Life Tracker', {
        body: body || 'Recordatorio',
        icon: '/icon-192.png',
        badge: '/icon-72.png',
        tag: tag || 'lt-scheduled',
        renotify: true,
        vibrate: [200, 100, 200],
        data: { url: url || '/dashboard.html' }
      });
    }, delay || 0);
  }

  if (event.data?.type === 'SKIP_WAITING') {
    self.skipWaiting();
  }
});
```